

Chapter 6

Lexical Semantics

Overview:

1. Lexeme, Lexicon, Senses, Lexical relations
2. WordNet (Lexical Database)
3. Word Sense Disambiguation (WSD)
4. Word Similarity

6.1 Semantic Analysis

- In the case of semantic analysis, a **computer understands the meaning of a text by analyzing the text as a whole** and not just looking at individual words.
- **The context in which a word is used is very important** when it comes to semantic analysis.
- It involves:
 - Identifying **word meanings** in context
 - Understanding **relationships between words**
 - Disambiguating **multiple meanings** of words
 - Extracting **facts, sentiments, or intent** from text

- Let's revisit the example: *“Does it all sound like a joke to you?”*
- While the word “joke” may be positive on its own, something sounding like a joke may not be so.
- Thus, the context here is derived by analyzing the whole sentence instead of isolated words.

What's an Orange?

- It **could be a fruit, a color, or a place!** To know the meaning of Orange in a sentence, we need to know the words around it.
- For example,
 - if the sentence talks about “**orange shirt**,” we are talking about the **color orange**.
 - If a sentence talks about **someone from Orange** wearing an **orange shirt** – we are talking about Orange, the **place**, and Orange, the **color**.
 - And, if we are talking about someone from Orange **eating an orange** while wearing an orange shirt – we are talking about the place, the color, and the **fruit**.
- You see, **the word on its own matters less**, and the **words surrounding it matter more for the interpretation**.
- A **semantic analysis algorithm needs to be trained with a larger corpus** of data to perform better.

Lexeme and Lexicon

- **Lexeme:**
 - A **lexeme** is a set of all the different forms a word can take.
 - **Examples of Lexemes**
 - The lexeme "run" includes the word forms: "run," "runs," "ran," "running."
 - The lexeme "child" includes the word forms: "child," "children."
 - The lexeme "go" includes the word forms: "go," "goes," "went," "gone," "going."
- **Lexicon:**
 - A **lexicon is the complete set of all lexemes** in a language, essentially functioning as the vocabulary of that language.
 - It includes not just the words themselves but also information about their meanings, forms, and usage.

Lexical relations

- **Lexical relations** refer to the **relationships between words or lexemes in a language**.
- These relationships **help to understand the meaning of words in relation to each other** and are essential for various linguistic tasks such as language learning, language processing, and computational linguistics.
- Some common types of **lexical relations**:
 - Synonymy, Antonymy, Hyponymy, Hypernymy,, Meronymy, Holonymy, Polysemy, Homonymy, Troponymy

Types of Lexical relations

1. Synonymy

- **Definition:** The relationship between **words that have similar or identical meanings.**
- **Example:** "Happy" and "joyful".

2. Antonymy

- **Definition:** The relationship between **words that have opposite meanings.**
- **Example:** "Hot" and "cold"

3. Hypernymy and Hyponymy

- **Definition:** The relationship between **a more general term (hypernym) and a more specific term (hyponym).**
- **Example:** "Bird" (hypernym) and "sparrow" (hyponym).

4. Holonymy and Meronymy

- **Definition:** The relationship between **a whole and its parts.**
- **Example:** "Car" (whole) and "wheel" (part)

5. Polysemy

- **Definition:** The relationship where **a single word has multiple related meanings**.
- **Example:** Head (body part, leader, top of something). All meanings share the idea of “top” or “front”

6. Homonymy

- **Definition:** The relationship where a **single word has multiple unrelated meanings**.
- **Example:** "Bat" (flying mammal) and "bat" (sports equipment).

7. Troponymy

- **Definition:** The relationship between **a more general verb and a more specific verb**.
- **Example:** "To walk" (general) and "to stroll" (specific).

8. Collocation

- **Definition:** The **tendency for certain words to co-occur more frequently** than would be expected by chance.
- **Example:** "Make a decision" and "take a risk".

9. Derivation

- **Definition:** The relationship between **a base word and words derived from it**.
- **Example:** "Happy" (base) and "happiness" (derived).

Hypernyms

- **Hypernyms gives us a more abstract term for a word.**

```
word = wordnet.synsets("Play")[0]

#Find more abstract term :
print(word.hypernyms())

[Synset('dramatic_composition.n.01')]
```

Hyponoms

- Hyponyms gives us a more specific term for a word.

```
word = wordnet.synsets("Play")[0]
```

```
#Find more specific term :  
word.hyponyms()
```

```
[Synset('grand_guignol.n.01'),  
 Synset('miracle_play.n.01'),  
 Synset('morality_play.n.01'),  
 Synset('mystery_play.n.01'),  
 Synset('passion_play.n.01'),  
 Synset('playlet.n.01'),  
 Synset('satyr_play.n.01'),  
 Synset('theater_of_the_absurd.n.01')]
```

Getting a name only

```
word = wordnet.synsets("Play")[0]  
  
#Get only name :  
print(word.lemmas()[0].name())
```

play

Synonyms

```
#Finding synonyms :  
  
#Empty list to store synonyms :  
synonyms = []  
  
for words in wordnet.synsets('Fun'):  
    for lemma in words.lemmas():  
        synonyms.append(lemma.name())  
  
synonyms
```

```
['fun',  
 'merriment',  
 'playfulness',  
 'fun',  
 'play',  
 'sport',  
 'fun',  
 'playfulness',  
 'fun']
```

Antonyms

```
#Finding antonyms :

#Empty list to store antonyms :
antonyms = []

for words in wordnet.synsets('Natural'):
    for lemma in words.lemmas():
        if lemma.antonyms():
            antonyms.append(lemma.antonyms()[0].name())

#Print antonyms :
antonyms

['unnatural', 'artificial', 'supernatural', 'sharp']
```

Synonyms and antonyms

```
#Finding synonyms and antonyms :

#Empty lists to store synonyms/antonyms :
synonyms = []
antonyms = []

for words in wordnet.synsets('New'):
    for lemma in words.lemmas():
        synonyms.append(lemma.name())
        if lemma.antonyms():
            antonyms.append(lemma.antonyms()[0].name())

#Print Lists :
print(synonyms)
print("\n")
print(antonyms)

['new', 'fresh', 'new', 'novel', 'raw', 'new', 'new', 'unexampled', 'new', 'new', 'newfangled', 'new', 'New', 'Modern', 'New',
'new', 'young', 'new', 'newly', 'freshly', 'fresh', 'new']

['old', 'worn']
```

6.2 WordNET

- **WordNet** is a large **lexical database** of the English language that groups words into sets of synonyms called **synsets** and records the **semantic relationships** between them.
- It is widely used in **Natural Language Processing (NLP)** and **computational linguistics**.
- The cognitive synonyms which are called **synsets** are presented in the database with lexical and semantic relations.
- For word “car”:
 - Synset: ['car', 'auto', 'automobile', 'machine', 'motorcar']

- **We can check how many different definitions of a word are available in Wordnet.**

```
from nltk.corpus import wordnet  
  
for words in wordnet.synsets("Fun"):  
    print(words)
```

```
Synset('fun.n.01')  
Synset('fun.n.02')  
Synset('fun.n.03')  
Synset('playfulness.n.02')
```

- **We can also check the meaning of those different definitions.**

```
#How many differnt meanings :  
for words in wordnet.synsets("Fun"):  
    for lemma in words.lemmas():  
        print(lemma)  
    print("\n")
```

```
Lemma('fun.n.01.fun')  
Lemma('fun.n.01.merriment')  
Lemma('fun.n.01.playfulness')
```

```
Lemma('fun.n.02.fun')  
Lemma('fun.n.02.play')  
Lemma('fun.n.02.sport')
```

```
Lemma('fun.n.03.fun')
```

```
Lemma('playfulness.n.02.playfulness')  
Lemma('playfulness.n.02.fun')
```

Finding all the details of a specific word

```
word = wordnet.synsets("Play")[0]
```

```
#Checking name :
```

```
print(word.name())
```

```
#Checking definition :
```

```
print(word.definition())
```

```
#Checking examples:
```

```
print(word.examples())
```

```
play.n.01
```

```
a dramatic work intended for performance by actors on a stage
```

```
['he wrote several plays but only one was produced on Broadway']
```

All details for all meanings of a word.

```
#Word meaning with definitions :  
for words in wordnet.synsets("Fun"):  
    print(words.name())  
    print(words.definition())  
    print(words.examples())  
  
    for lemma in words.lemmas():  
        print(lemma)  
    print("\n")
```

```
fun.n.01  
activities that are enjoyable or amusing  
['I do it for the fun of it', 'he is fun to have around']  
Lemma('fun.n.01.fun')  
Lemma('fun.n.01.merriment')  
Lemma('fun.n.01.playfulness')
```

```
fun.n.02  
verbal wit or mockery (often at another's expense but not to be taken seriously)  
['he became a figure of fun', 'he said it in sport']  
Lemma('fun.n.02.fun')  
Lemma('fun.n.02.play')  
Lemma('fun.n.02.sport')
```

```
fun.n.03  
violent and excited activity  
['she asked for money and then the fun began', 'they began to fight like fun']  
Lemma('fun.n.03.fun')
```

```
playfulness.n.02  
a disposition to find (or make) causes for amusement  
['her playfulness surprised me', 'he was fun to be with']  
Lemma('playfulness.n.02.playfulness')  
Lemma('playfulness.n.02.fun')
```

Key Tasks in Semantic Analysis:

Task

Description

Word Sense

Disambiguation (WSD)

Determines which sense (meaning) of a word is used in a sentence. Example: “*bank*” → riverbank or financial bank?

Named Entity

Recognition (NER)

Identifies entities like people, organizations, locations.

Semantic Role Labeling (SRL)

Identifies the role played by each word in a sentence (who did what to whom, when, where).

Coreference Resolution

Finds which words refer to the same thing (e.g., “John said he will come” → “he” = “John”).

Lexical Semantics

Deals with word meanings and relationships (synonyms, antonyms, hyponyms).

Compositional Semantics

Combines word meanings based on grammar to derive sentence meaning.

6.3 Word Sense

- In Natural Language Processing (NLP), **word senses refer to the distinct meanings of a word or phrase in different contexts.**
- Words often have multiple senses, and **disambiguating these senses is crucial** for understanding and processing language accurately. This process is known as Word Sense Disambiguation (WSD).

Examples:

- The word "bat" can mean a flying mammal or a piece of sports equipment.
- The word "bank" can mean a financial institution or the side of a river. Both meanings are related to the idea of a 'place' or 'edge'.

6.4 Word Sense Disambiguation (WSD)

- WSD is the **process of identifying which sense of a word is used in a given context when the word has multiple meaning.**
- It's a crucial task in natural language processing (NLP) because many words are polysemous, meaning they have **multiple senses.**
- Accurate WSD enhances the performance of various NLP applications, including machine translation, information retrieval, and text summarization.
- **Example:** In the sentence "He went to the bank to fish," WSD helps determine that "bank" refers to the side of a river, not a financial institution.

Methods of WSD

1. Use of Machine learning: Supervised and unsupervised
2. Use of Knowledge-based methods: Lesk algorithm, WordNet-based methods
3. Hybrid methods: combination of both of the above methods

Lesk Algorithm:

- This algorithm **measures the overlap between the context of a word** and the definitions of its senses in a dictionary.
- The sense with the highest overlap is chosen.

WordNet-based Methods:

- WordNet, a lexical database, provides definitions, examples, and relations between words.
- Algorithms use WordNet to find the most relevant sense based on context.

Example:

Consider the word "bank" in two different sentences:

- "He deposited money in the bank."
- "He sat on the bank of the river."

To disambiguate the word "bank":

1. **Feature Extraction:** Extract words surrounding "bank" (e.g., "deposited", "money", "sat", "river").
2. **Supervised Learning:** Use a pre-trained model that has learned the senses of "bank" from labeled examples to predict the sense in each sentence.
3. **Knowledge-based:** Use WordNet to check the definitions and contexts of "bank" related to financial institutions and river edges, and choose the sense that best matches the context.

6.5 Word similarity

- **Word similarity** in Natural Language Processing (NLP) **refers to the measurement of how alike two words are in terms of their meanings** or usage in language.
- Word similarity is a fundamental concept used in various NLP applications, such as information retrieval, machine translation, text summarization, and more.
- There are several methods and techniques to compute word similarity, broadly categorized into
 - semantic similarity and
 - distributional similarity.

Types of Word similarity

- Semantic Similarity

- **Definition:** Measures how similar the meanings of two words are based on lexical and semantic resources.
- **Example:** "car" and "automobile" are semantically similar because they both refer to the same concept.

- Distributional Similarity

- **Definition:** Based on the distributional hypothesis, which states that words appearing in similar contexts tend to have similar meanings.
- **Example:** "dog" and "pet" may be considered similar because they often appear in similar contexts (e.g., "The dog is a loyal pet").

Finding the similarity between words.

- Finding the similarity ratio between words using Wordnet.

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("boat","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))  
  
0.9090909090909091
```

Similarity ratio between two words

- Using WordNet

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("boat","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))
```

0.9090909090909091

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("bike","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))
```

0.6956521739130435

Bag of Words:

- It is a **method of extracting essential features from raw text** so that we can use it for machine learning models.
- We call it “**Bag**” of words because we discard the order of occurrences of words.
- A bag of words model converts the raw text into words, and it also counts the frequency for the words in the text.
- In summary, a bag of words is a collection of words that represent a sentence along with the word count where the order of occurrences is not relevant.



Applications:

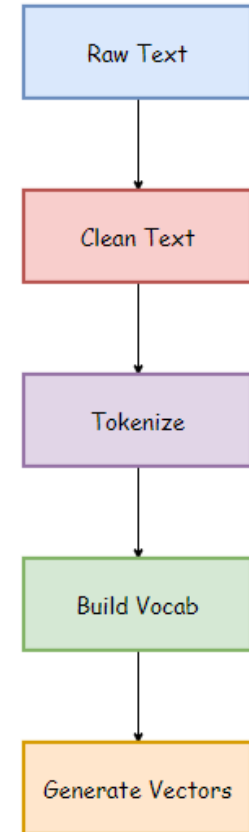
1. Natural language processing.
2. Information retrieval from documents.
3. Classifications of documents.

Limitations:

1. **Semantic meaning:** It does not consider the semantic meaning of a word. It ignores the context in which the word is used.
2. **Vector size:** For large documents, the vector size increase, which may result in higher computational time.
3. **Preprocessing:** In preprocessing, we need to perform data cleansing before using it.

Structure of a bag of words

- **Raw Text:** This is the original text on which we want to perform analysis.
- **Clean Text:** Since our raw text contains some unnecessary data like punctuation marks and stopwords, so we need to clean up our text. Clean text is the text after removing such words.
- **Tokenize:** Tokenization represents the sentence as a group of tokens or words.
- **Building Vocab:** It contains total words used in the text after removing unnecessary data.
- **Generate Vocab:** It contains the words along with their frequencies in the sentences.



Example

Sentences are:

1. Jim and Pam traveled by the bus.
2. The train was late.
3. The flight was full. Traveling by flight is expensive.

1. Jim and Pam traveled by the bus.
2. The train was late.
3. The flight was full. Traveling by flight is expensive.

a. Creating a basic structure:

Sentence 1	Sentence2	Sentence 3
Jim	The	The
and	train	flight
Pam	was	was
travelled	late	full
by		Travelling
the		by
bus		flight
		is
		expensive

b. Words with frequencies:

1. Jim and Pam traveled by bus.
2. The train was late.
3. The flight was full. Traveling by flight is expensive.

Sentence1	Count	Sentence2	Count	Sentence3	Count
Jim	1	The	1	The	1
and	1	train	1	flight	2
Pam	1	was	1	was	1
travelled	1	late	1	full	1
by	1			Travelling	1
the	1			by	1
bus	1			is	1
				expensive	1

c. Combining all the words:

Sentence	Frequency
and	1
bus	1
by	2
expensive	1
flight	2
full	1
is	1
jim	1
late	1
pam	1
the	3
train	1
travelled	1
travelling	1
was	1

d. Final model:

	and	bus	by	exp ensi ve	fligh t	full	is	jim	Late	pam	The	train	trav elle d	trav ellin g	was
S-1	1	1	1	0	0	0	0	1	0	1	1	0	1	0	0
S-2	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1
S-3	0	0	1	1	2	1	1	0	0	0	1	0	0	1	1

Python Implementation

```
#Import required libraries :  
from sklearn.feature_extraction.text import CountVectorizer
```

```
#Text for analysis :  
sentences = ["Jim and Pam travelled by the bus:",  
             "The train was late",  
             "The flight was full.Travelling by flight is expensive"]
```

```
#Create an object :  
cv = CountVectorizer()
```

```
#Generating output for Bag of Words :  
B_O_W = cv.fit_transform(sentences).toarray()
```

```
#Total words with their index in model :
```

```
print(cv.vocabulary_)  
print("\n")
```

```
{'jim': 7, 'and': 0, 'pam': 9, 'travelled': 12, 'by': 2, 'the': 10, 'bus': 1, 'train': 11, 'was': 14, 'late': 8, 'flight': 4,  
'full': 5, 'travelling': 13, 'is': 6, 'expensive': 3}
```

```
#Features :
```

```
print(cv.get_feature_names())  
print("\n")
```

```
['and', 'bus', 'by', 'expensive', 'flight', 'full', 'is', 'jim', 'late', 'pam', 'the', 'train', 'travelled', 'travelling', 'was']
```

```
#Show the output :  
print(B_O_W)
```

```
[[1 1 1 0 0 0 0 1 0 1 1 0 1 0 0]  
 [0 0 0 0 0 0 0 0 1 0 1 1 0 0 1]  
 [0 0 1 1 2 1 1 0 0 0 1 0 0 1 1]]
```

```
[[1 1 1 0 0 0 0 1 0 1 1 0 1 0 0]  
 [0 0 0 0 0 0 0 0 1 0 1 1 0 0 1]  
 [0 0 1 1 2 1 1 0 0 0 1 0 0 1 1]]
```

TF-IDF

- TF-IDF stands for **Term Frequency — Inverse Document Frequency**, which is a **scoring measure** generally used in information retrieval (IR) and summarization.
- The TF-IDF score **shows how important** or relevant a term is in a given document.

The intuition behind TF and IDF:

- If a particular word appears multiple times in a document, then it might have higher importance than the other words that appear fewer times (TF).
- At the same time, if a particular word appears many times in a document, but it is also present many times in some other documents, then maybe that word is frequent, so we cannot assign much importance to it (IDF).

Example:

- **Sentence 1:** This is the first document.
- **Sentence 2:** This document is the second document.

TF and IDF

- Term Frequency

$$TF = \frac{\textit{Frequency of the word in the sentence}}{\textit{Total number of words in the sentence}}$$

- Inverse Document Frequency

$$IDF = \log \left(\frac{\textit{Total number of sentences}}{\textit{Number of sentences containing that word}} \right)$$

- TF-IDF

$$TFIDF = TF * IDF$$

Sentence 1: This is the first document.

Sentence 2: This document is the second document.

a. Represent the words of the sentences in the table.

Sentence 1	Sentence 2
This	This
is	document
the	is
first	the
document	second
	document

Sentence 1: This is the first document.

Sentence 2: This document is the second document.

b. Displaying the frequency of words.

Sentence 1	TF	Sentence 2	TF
This	1	This	1
is	1	document	2
the	1	is	1
first	1	the	1
document	1	second	1

Sentence 1: This is the first document.

Sentence 2: This document is the second document.

c. Calculating TF using a formula.

$$TF = \frac{\text{Frequency of the word in the sentence}}{\text{Total number of words in the sentence}}$$

Sentence 1	TF	Sentence 2	TF
This	$\frac{1}{5} = 0.20$	This	$\frac{1}{6} = 0.166$
is	$\frac{1}{5} = 0.20$	document	$\frac{1}{3} = 0.33$
the	$\frac{1}{5} = 0.20$	is	$\frac{1}{6} = 0.166$
first	$\frac{1}{5} = 0.20$	the	$\frac{1}{6} = 0.166$
document	$\frac{1}{5} = 0.20$	second	$\frac{1}{6} = 0.166$

- **IDF: Inverse Document Frequency**

$$IDF = \frac{\text{Total number of sentences}}{\text{Number of sentences containing that word}}$$

Sentence 1	IDF	Sentence 2	IDF
This	$2/2 = 1$	This	$2/2 = 1$
is	$2/2 = 1$	document	$2/2 = 1$
the	$2/2 = 1$	is	$2/2 = 1$
first	$2/1 = 2$	the	$2/2 = 1$
document	$2/2 = 1$	second	$2/1 = 2$

e. Calculating TF-IDF.

TF-IDF is the multiplication of TF*IDF

Word	TF-IDF 1	TF-IDF2
This	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
is	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
the	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
first	$0.20 * 2 = 0.40$	$0 * 2 = 0$
document	$0.20 * 1 = 0.20$	$0.333 * 1 = 0.33$
second	$0 * 2 = 0$	$0.166 * 2 = 0.33$

- In this case, notice that the import words that discriminate both the sentences are “first” in sentence-1 and “second” in sentence-2 as we can see, those words have a relatively higher value than other words.
- However, there are many variations for smoothing out the values for large documents. The most common variation is to use a log value for TF-IDF. Let’s calculate the TF-IDF value again by using the new IDF value.

$$IDF = \log \left(\frac{\text{Total number of sentences}}{\text{Number of sentences containing that word}} \right)$$

f. Calculating IDF value using log.

Word	IDF 1	IDF 2
This	$\log(1) = 0$	$\log(1) = 0$
is	$\log(1) = 0$	$\log(1) = 0$
the	$\log(1) = 0$	$\log(1) = 0$
first	$\log(2) = 0.693$	$\log(2) = 0.693$
document	$\log(1) = 0$	$\log(1) = 0$
second	$\log(2) = 0.693$	$\log(2) = 0.693$

g. Calculating TF-IDF.

As seen above, “first” and “second” values are important words that help us to distinguish between those two sentences.

- Now that we saw the basics of TF-IDF. Next, we are going to use the sklearn library to implement TF-IDF in Python. A different formula calculates the actual output from our program. First, we will see an overview of our calculations and formulas, and then we will implement it in Python.

Word	TF-IDF 1	TF-IDF 2
This	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
is	$0.20 * \log(1) = 0$	$0.333 * \log(1) = 0$
the	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
first	$0.20 * \log(2) = 0.13$	$0 * \log(2) = 0$
document	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
second	$0 * \log(2) = 0$	$0.166 * \log(2) = 0.13$

Actual Calculations:

a. Term Frequency (TF):

Sentence 1	TF	Sentence 2	TF
This	1	This	1
is	1	document	2
the	1	is	1
first	1	the	1
document	1	second	1

b. Inverse Document Frequency (IDF):

$$IDF = \log \left(\frac{\text{Total number of sentences} + 1}{\text{Number of sentences containing that word} + 1} \right)$$

Word	IDF 1	IDF 2
This	$\log(3/3) = 0$	$\log(3/3) = 0$
is	$\log(3/3) = 0$	$\log(3/3) = 0$
the	$\log(3/3) = 0$	$\log(3/3) = 0$
first	$\log(3/2) = 0.40$	$\log(3/2) = 0.40$
document	$\log(3/3) = 0$	$\log(3/3) = 0$
second	$\log(3/2) = 0.40$	$\log(3/2) = 0.40$

c. Calculate TF-IDF values:

$$TF - IDF = (TF) * (IDF + 1)$$

Word	TF-IDF 1	TF-IDF 2
This	$1 * (0 + 1) = 1$	$1 * (0 + 1) = 1$
is	$1 * (0 + 1) = 1$	$1 * (0 + 1) = 1$
the	$1 * (0 + 1) = 1$	$1 * (0 + 1) = 1$
first	$1 * (0.40 + 1) = 1.40$	$0 * (0.40 + 1) = 0$
document	$1 * (0 + 1) = 1$	$2 * (0 + 1) = 2$
second	$0 * (0.40 + 1) = 0$	$1 * (0.40 + 1) = 1.40$

Python Implementation:

```
#Import required libraries :
from sklearn.feature_extraction.text import TfidfVectorizer

#Sentences for analysis :
sentences = ['This is the first document','This document is the second document']

#Create an object :
vectorizer = TfidfVectorizer(norm = None)

#Generating output for TF_IDF :
X = vectorizer.fit_transform(sentences).toarray()

#Total words with their index in model :
print(vectorizer.vocabulary_)
print("\n")

#Features :
print(vectorizer.get_feature_names())
print("\n")

#Show the output :
print(X)
```

`{'this': 5, 'is': 2, 'the': 4, 'first': 1, 'document': 0, 'second': 3}`

`['document', 'first', 'is', 'second', 'the', 'this']`

<code>[[1.</code>	<code>1.40546511</code>	<code>1.</code>	<code>0.</code>	<code>1.</code>	<code>1.</code>	<code>]</code>
<code>[2.</code>	<code>0.</code>	<code>1.</code>	<code>1.40546511</code>	<code>1.</code>	<code>1.</code>	<code>]]</code>

References:

- <https://towardsai.net/p/nlp/natural-language-processing-nlp-with-python-tutorial-for-beginners-1f54e610a1a0>

Methods to compute word similarity

- Cosine Similarity
- Jaccard similarity
- Latent Semantic indexing (LSI)

Cosine similarity :

- **calculates the cosine of the angle** between two documents
- **0 indicates exactly opposite, 1 indicates exactly same**, and while others are intermediate values

$$\text{Cosine similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- **Where,**
- **A= TFIDF of document A**
- **B=TFIDF of document B**

Assignment:

- Given below is the TF-IDF of two documents, compute the cosine similarity of two documents:
- Doc A: “The car is driven on the road.”
- Doc B: “The truck is driven on the highway.”

	car	driven	highway	is	on	road	the	truck
Doc1	0.424717	0.30219	0.000000	0.30219	0.30219	0.424717	0.60438	0.000000
Doc2	0.000000	0.30219	0.424717	0.30219	0.30219	0.000000	0.60438	0.424717

Named Entity Recognition (NER):

- Named entity recognition can automatically scan entire articles **and pull out some fundamental entities** like people, organizations, places, date, time, money, and GPE discussed in them.
- **Use-Cases:**
 - Content classification for news channels.
 - Summarizing resumes.
 - Optimizing search engine algorithms.
 - Recommendation systems
 - Customer support

Commonly used types of named entity:

Named Entity Type	Example
ORGANIZATION	WHO
PERSON	President Obama
LOCATION	Mount Everest
DATE	2020-07-10
TIME	12:50 P.M.
MONEY	One Million Dollars
PERCENT	98.24%
FACILITY	Washington Monument
GPE	North West America

There are two options :

1. binary = True

When the binary value is True, then it will only show whether a particular entity is named entity or not. It will not show any further details on it.

2. binary = False

When the binary value equals False, it shows in detail the type of named entities.

```

#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words,binary=True)

print(N_E_R)

#To visualize :
N_E_R.draw()

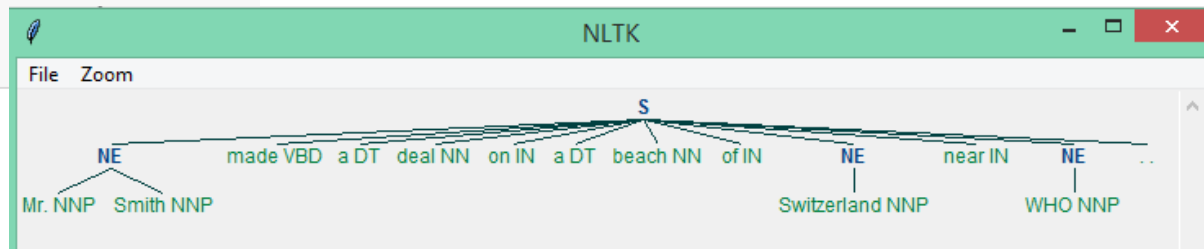
```

- Our graph does not show what type of named entity it is. It only shows whether a particular word is named entity or not.

```

(S
 (NE Mr./NNP Smith/NNP)
 made/VBD
 a/DT
 deal/NN
 on/IN
 a/DT
 beach/NN
 of/IN
 (NE Switzerland/NNP)
 near/IN
 (NE WHO/NNP)
 ./.)

```



Graph example of when a binary value is True.

```

#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words,binary=False)
print(N_E_R)

#To visualize :
N_E_R.draw()

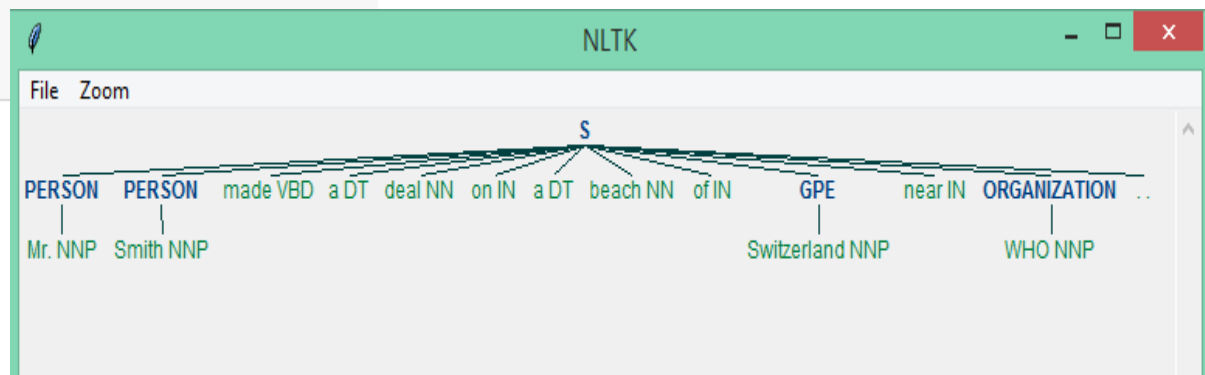
```

```

(S
 (PERSON Mr./NNP)
 (PERSON Smith/NNP)
 made/VBD
 a/DT
 deal/NN
 on/IN
 a/DT
 beach/NN
 of/IN
 (GPE Switzerland/NNP)
 near/IN
 (ORGANIZATION WHO/NNP)
 ./.)

```

- Our graph now shows what type of named entity it is.



Graph showing the type of named entities when a binary value equals false.

End of chapter 6